FRAMEWORK EXPLANATION DOCUMENT

Selenium GUI Elements Automation

Design, Architecture, and Code Walkthrough | June 2026

1. Framework Overview

This framework automates the GUI elements on https://testautomationpractice.blogspot.com using Selenium WebDriver 4.4.0. It follows industry-standard design patterns to keep the code maintainable, readable, and reusable. The framework supports Chrome and Firefox browsers, generates a rich HTML report with embedded screenshots, and is fully managed by Maven.

2. Framework Architecture

The framework is built on three core layers:

Layer	Class / File	Responsibility
Configuration	config.properties	[redacted]
Page Layer (POM)	GuiElementsPage.java	Contains all element locators and interaction methods. Tests never use raw WebDriver here.
Test Layer	GuiElementsTest.java	Contains @Test methods with assertions. Calls Page layer methods.
Base Layer	BaseTest.java	Handles @BeforeSuite, @AfterSuite, driver init, report setup, screenshot on failure.
Utility Layer	ExtentReportManager, ScreenshotUtil	Manages HTML reporting and screenshot capture independently.

3. Design Patterns Used

3.1 Page Object Model (POM)

POM is the core design pattern. Every webpage is represented by a Java class. All element locators (By) and action methods live inside this class. Test classes never use driver.findElement() directly — they only call methods from the page class.

Benefits:

- If a locator changes, you update it in one place only
- Test methods are short, readable, and focused on behavior
- Reusable methods across multiple test classes

```
// GuiElementsPage.java — Page Object
public class GuiElementsPage {
    private WebDriver driver;

    // Locators defined once
    private By nameField  = By.id("name");
    private By maleRadio  = By.id("male");

    // Action method — test calls this, not driver.findElement
    public void enterName(String name) {
        driver.findElement(nameField).clear();
        driver.findElement(nameField).sendKeys(name);
    }

    public void selectGender(String gender) {
        if (gender.equalsIgnoreCase("Male"))
            driver.findElement(maleRadio).click();
    }
}
```

3.2 Singleton Pattern — ConfigReader

ConfigReader uses the Singleton pattern to ensure the config.properties file is loaded only once throughout the entire test run. All classes share the same instance.

```
public class ConfigReader {
    private static ConfigReader instance;   // single instance
    private Properties properties;

    private ConfigReader() { loadProperties(); }

    public static ConfigReader getInstance() {
        if (instance == null)
            instance = new ConfigReader();   // created only once
        return instance;
    }
```

```
    public String getBrowser()  { return properties.getProperty("browser"); }
    public String getAppUrl()   { return properties.getProperty("app.url"); }
}
```

3.3 ThreadLocal WebDriver — DriverManager

DriverManager uses ThreadLocal<WebDriver> to store the browser instance. This makes the framework thread-safe and ready for parallel test execution — each thread gets its own independent WebDriver.

```
public class DriverManager {
    // Each thread gets its own WebDriver instance
    private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

    public static void initDriver() {
        WebDriverManager.chromedriver().setup();
        driver.set(new ChromeDriver());
    }

    public static WebDriver getDriver() {
        return driver.get();  // returns THIS thread's driver
    }

    public static void quitDriver() {
        driver.get().quit();
        driver.remove();      // clean up to prevent memory leaks
    }
}
```

4. Selenium APIs Used

4.1 WebDriver & Basic Interactions

API	Used For
driver.get(url)	Navigate to the application URL
element.sendKeys(text)	Type text into Name, Email, Phone, Address fields
element.click()	Click radio buttons, checkboxes, alert buttons
element.clear()	Clear field before entering new value
element.getAttribute("value")	Read field value for assertion
element.isSelected()	Verify radio button / checkbox is checked

4.2 Select Class — Dropdowns

```
// Country dropdown
Select select = new Select(driver.findElement(By.id("country")));
select.selectByVisibleText("India");   // select by display text
select.selectByValue("india");         // select by option value attribute
select.selectByIndex(2);               // select by position
```

4.3 Actions Class — Mouse & Keyboard

```
Actions actions = new Actions(driver);

// Mouse Hover
actions.moveToElement(element).perform();

// Double Click
actions.doubleClick(element).perform();

// Drag and Drop
actions.dragAndDrop(sourceElement, targetElement).perform();

// Slider — send arrow keys to range input
actions.click(slider).sendKeys(Keys.ARROW_RIGHT).perform();
```

4.4 Alert Handling

```
// Wait for alert then interact with it
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
Alert alert = wait.until(ExpectedConditions.alertIsPresent());

String text = alert.getText();    // read the alert message
alert.accept();                    // click OK
alert.dismiss();                   // click Cancel
```

4.5 Explicit Waits

The framework uses explicit waits (WebDriverWait + ExpectedConditions) instead of Thread.sleep(). This makes tests faster and more reliable.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));

// Wait for element to be visible before interacting
WebElement el = wait.until(
    ExpectedConditions.visibilityOfElementLocated(By.id("name"))
);

// Wait for element to be clickable (visible + enabled)
WebElement btn = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("dblClkBtn"))
);
```

4.6 JavascriptExecutor — Scrolling

```
// Scroll an element into the centre of the viewport
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript(
    "arguments[0].scrollIntoView({behavior:'smooth', block:'center'});",
    element
);

// Scroll down by pixel amount
js.executeScript("window.scrollBy(0, 400);");
```

5. Reporting and Screenshots

5.1 ExtentReports

The framework uses ExtentReports 5.1.1 (Spark reporter) to generate a dark-themed HTML report. The report is created by ExtentReportManager in @BeforeSuite and flushed to disk in @AfterSuite.

```
// BaseTest.java — report lifecycle
@BeforeSuite
public void beforeSuite() {
    ExtentReportManager.initReports();  // creates the HTML file
}

@AfterMethod
public void afterMethod(ITestResult result) {
    if (result.getStatus() == ITestResult.FAILURE) {
        String screenshot = ScreenshotUtil.capture(driver, result.getName());
        ExtentReportManager.getTest().fail("FAILED", screenshot);
    }
}

@AfterSuite
public void afterSuite() {
    ExtentReportManager.flush();  // writes all results to disk
}
```

5.2 ScreenshotUtil

Screenshots are captured at key steps and on test failure using TakesScreenshot. Files are saved to the screenshots/ folder with a timestamp in the filename.

```
public static String capture(WebDriver driver, String testName) {
    TakesScreenshot ts = (TakesScreenshot) driver;
    File src  = ts.getScreenshotAs(OutputType.FILE);
    String path = "screenshots/" + testName + "_" +
                  new SimpleDateFormat("yyyyMMdd_HHmmss")
                      .format(new Date()) + ".png";
    FileUtils.copyFile(src, new File(path));
    return path;
}
```

6. Project Structure

File / Folder	Description
pom.xml	Maven build file — all dependency versions defined here

testng.xml	TestNG suite file — controls which tests run and in what order
GuiElementsPage.java	Page Object — all locators and browser interaction methods
GuiElementsTest.java	34 @Test methods with TestNG assertions and ExtentReport logging
BaseTest.java	Setup/teardown: opens browser before each test, quits after
ExtentReportManager.java	Singleton — creates, manages, and flushes the HTML report
ScreenshotUtil.java	Captures and saves .png screenshots with timestamps
log4j2.xml	Logging configuration — suppresses noisy Selenium console output
screenshots/	Auto-created folder where all screenshots are saved
test-output/	Auto-created folder containing ExtentReport.html

7. How to Run the Tests

Step	Action
1	Open Eclipse → File → Import → Maven → Existing Maven Projects → select the project folder
2	Right-click project → Maven → Update Project → Force Update → OK
3	Right-click testng.xml → Run As → TestNG Suite (OR: run 'mvn clean test' in terminal)
4	After run completes, open test-output/ExtentReport.html in Chrome to view results